The Agent Loop Guide

Stop prompting your agents one task at a time. Start designing loops that do it for you.

CONTENTS

01

What is Agent Looping?

For the last two years, we prompted agents one task at a time. We asked, they answered, we asked again. We were the loop. That is starting to change.

Looping is a setup you build. Instead of driving every step yourself, you define a cycle: discover, plan, execute, check. The agent runs it until the goal is met. You stop prompting each step. The system repeats the cycle for you.

> Don't prompt your agents. Design loops that prompt agents. Your job shifts from giving instructions to designing the conditions under which the right instructions get generated automatically.

At its simplest, looping is **one agent working on itself:** it researches, drafts, checks the draft against a goal, fixes what is weak, then runs that cycle again until the work clears your requirements. You set the bar. The agent reaches it.

The bigger version is **a fleet looping.** An orchestrator agent takes your goal, breaks it into pieces, hands each piece to a specialist, and those specialists hand smaller jobs to their own subagents. The whole tree keeps running through discovery, planning, execution, and verification until the goal is met.

One agent looping is like a person redrafting their own work. A fleet looping is a whole team running a project end to end, with you setting the goal and reviewing only what matters.

02

Two Types of Loops

Not all loops are the same shape. The most important distinction is whether the loop ends or runs forever.

TYPE ONE

One-Off Loop

Babysits a task and terminates once the goal is reached. You define a finish condition. It works until that condition is true, then stops.

- Babysit the CI pipeline for this PR. Fix failures as they come up. Stop when it merges.
- Find out-of-date docs in this repo. Spin up a subagent to fix each one. Stop when none remain.
- Research, draft, and iterate this report until it clears my eval rubric.

TYPE TWO

Durable Loop

A cron job that runs forever. Valuable when you have a regular stream of new things arriving that need processing on a schedule.

- Every hour: scan GitHub repos for new contributor issues and triage with labels.
- Every night: scan social posts and consolidate viewpoints into OPINIONS.md.
- Every week: review open support tickets and surface the top three patterns.

> **One-off:** when you have a discrete goal with a clear done state: PR merged, report approved, bug fixed.
>
> **Durable:** when you have a recurring stream of new input (user feedback, production alerts, security scans, social content) that needs regular processing.

03 + 04

Open vs. Closed + the Fleet

MODE A

Open Loop

- Wide solution space: the agent can explore paths you didn't fully spec
- Goal is fixed but the route is not; no rigid step order
- Burns more tokens; needs a real budget or tight domain
- Risk of slop without a clearly defined success bar

EXPLORATORY

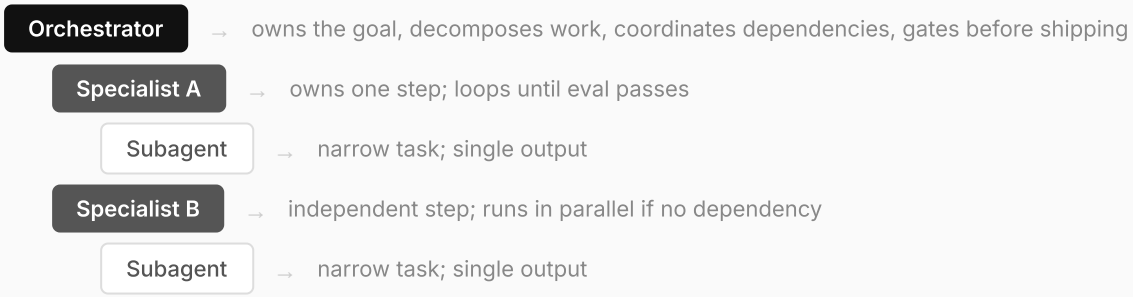
MODE B

Closed Loop

- Human-designed path: clear goal, defined steps, eval at each step, hard stop or handback
- Agents loop, but inside a framework you built
- Gets better every run: each pass feeds data into the next
- Runs on a normal budget because the path is tight

BOUNDED / RECOMMENDED

Fleet Architecture



The orchestrator reviews output against the original goal and only ships when it clears. This is the eval gate, the thing that keeps the whole fleet from becoming a slop machine.